

State Machine Engine -- Requirements Specification

1. Problem Statement

Order lifecycle rules keep changing. Marketing wants to add a "pending review" step for high-value orders. Operations wants to prevent cancellation after shipping. Compliance needs every state transition logged with a reason.

Currently, lifecycle logic is scattered across services as if/elif chains. Every change requires finding all the relevant branches, updating them consistently, and re-testing the full flow.

You are building the State Machine Engine -- a Python library that lets developers define order lifecycle rules as explicit states and transitions, without touching transition logic.

A state machine defines named states and the events that move between them:

```
sm = StateMachine(initial="draft")

sm.transition(
    "submit",
    from_state="draft",
    to_state="submitted",
    guard='order.total > 0 AND customer.verified == true',
)

sm.transition(
    "cancel",
    from_state=["draft", "submitted", "processing"],
    to_state="cancelled",
)

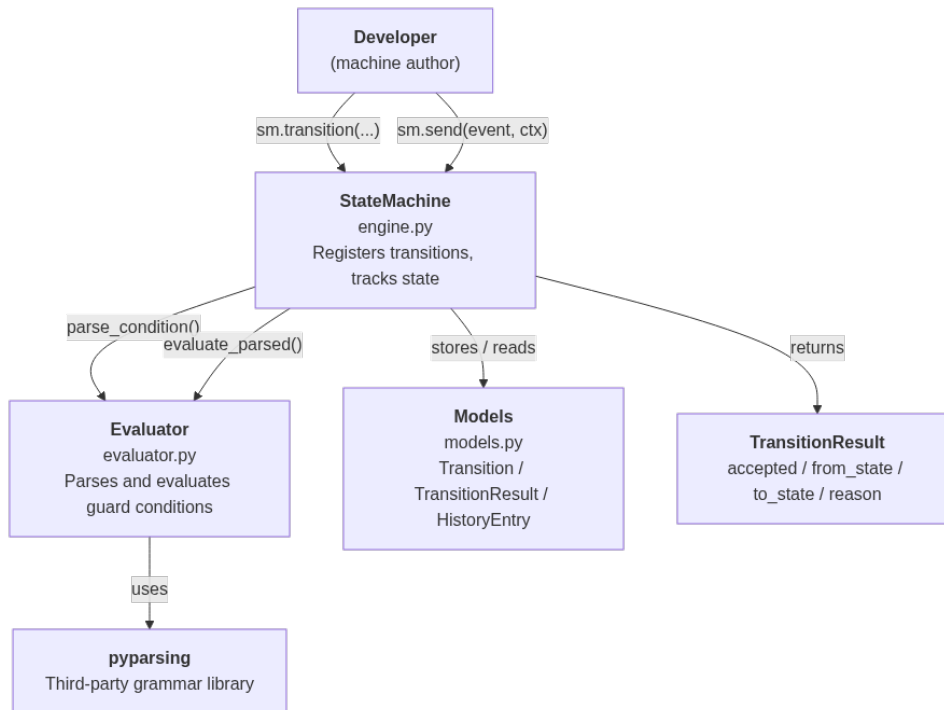
context = {
    "customer": {"verified": True},
    "order": {"total": 150},
}

result = sm.send("submit", context)
# result.accepted == True
# result.to_state == "submitted"
# sm.current_state == "submitted"
```

The engine tracks the current state and decides whether an event is valid, whether its guard condition passes, and whether its action succeeds -- before updating the state. If anything

fails, the state does not change.

Component Overview



2. Overview

A Python library for defining named state transitions and evaluating them against runtime data. Transitions consist of a source state, a destination state, an optional string-based guard condition, and an optional callable action.

Design goals:

- Guard conditions are human-readable strings, not Python code
 - Guard conditions are validated at transition registration time, not at runtime
 - State never changes unless the guard passes and the action succeeds
 - Full history of accepted transitions is always available
-

3. Current Implementation State

A partial implementation exists. The following is complete and working:

- `StateMachine.__init__(initial)` -- sets initial and current state
- `StateMachine.transition(name, from_state, to_state)` -- registers a transition (no guard, no action, `from_state` must be a single string)
 - `StateMachine.send(event, context)` -- finds matching transition for current state; returns `TransitionResult`; no guard evaluation, no action, no history
- `Transition`, `TransitionResult`, `HistoryEntry` data classes defined

The following is NOT yet implemented and must be completed:

Missing Feature	Impact
Guard condition evaluation	Guards are registered but never evaluated -- all transitions always accepted
<code>from_state</code> as a list	Multi-source transitions raise <code>TypeError</code>
Action execution	action is stored but never called
History tracking	<code>sm.history</code> always returns <code>[]</code>
<code>can()</code> method	Not implemented
<code>reset()</code> method	Not implemented
Duplicate transition detection	Registering the same (name, <code>from_state</code>) pair twice silently overwrites
Action error handling	An action exception bubbles up uncaught
<code>states</code> property	Not implemented

4. Core Concepts

States

States are named string labels. They do not need to be declared explicitly -- the engine infers all known states from the initial argument and the `from_state` / `to_state` values of registered transitions.

Events

An event is a named trigger sent to the machine via `sm.send(event, context)`. If a registered transition matches the current state and event name, and the guard passes, the transition fires.

Transitions

A transition maps an event from one or more source states to a destination state. It may optionally carry a guard condition and an action.

Guard Conditions

A guard is a string expression evaluated against the runtime context dict. If the guard evaluates to False, the transition is rejected and the state does not change. The guard language is identical to the Rules Engine condition language -- the same grammar, the same field access syntax, the same operators.

Actions

An action is a Python callable invoked with the context dict when a transition fires, before the state is updated. If the action raises any exception, the state is not updated and the transition is rejected.

5. Guard Condition Language

Guard conditions use the same expression language as the Rules Engine:

Operators

Category	Operators
Comparison	>, =, <=, ==, !=
Logical	AND, OR, NOT
Membership	IN, NOT IN

Literal Value Types

Type	Examples
Integer / Float	18, 5000, 99.95
String (quoted)	"active", "MY", 'gold'
Boolean	true, false
Null	null
List	["MY", "SG", "ID"], [1, 2, 3]

Field Access

Conditions reference context data using dot notation. There is no depth limit on nesting.

```
order.total
customer.verified
```

```
order.shipping.address.city
```

If any segment is missing or non-traversable, the evaluator raises `EvaluationError`.

Operator Precedence

Highest to lowest:

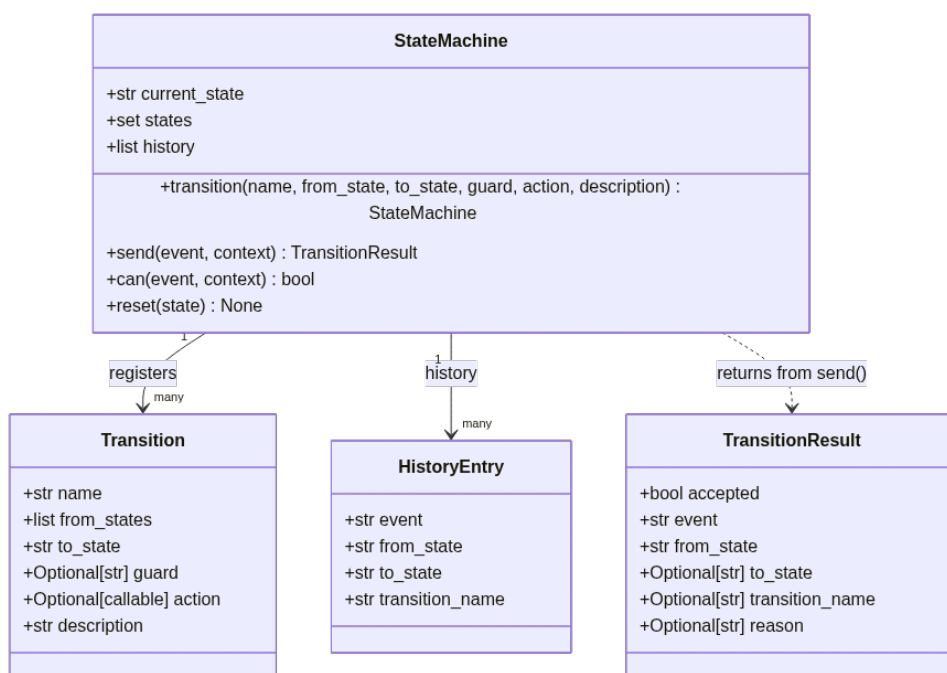
Level	Operator	Associativity
1	NOT	Right
2	AND	Left
3	OR	Left

A OR B AND C parses as A OR (B AND C).

Short-Circuit Evaluation

- A AND B -- if A is false, B is not evaluated
- A OR B -- if A is true, B is not evaluated

6. Data Models



Transition

Field	Type	Required	Default	Description
name	str	Yes	--	Event name that triggers this transition
from_states	list[str]	Yes	--	Source states; stored as list regardless of input
to_state	str	Yes	--	Destination state
guard	Optional[str]	No	None	Guard condition expression
action	Optional[Callable[[dict], Any]]	No	None	Called with context dict if guard passes
description	str	No	""	Human-readable documentation

TransitionResult

Returned by sm.send().

Field	Type	Description
accepted	bool	True if state changed
event	str	The event name that was sent
from_state	str	State at time of send()
to_state	Optional[str]	New state if accepted; None if rejected
transition_name	Optional[str]	Name of matching transition; None if no match found
reason	Optional[str]	Rejection reason if not accepted; None if accepted

HistoryEntry

One entry per accepted transition, in order.

Field	Type	Description
event	str	Event name
from_state	str	State before transition
to_state	str	State after transition
transition_name	str	Matching transition name

7. Engine Behaviour

7.1 Transition Registration

```
sm.transition(  
    name,  
    *,  
    from_state,      # str or list[str]  
    to_state,  
    guard=None,  
    action=None,  
    description="",  
)
```

- from_state is normalised to list[str] internally regardless of input type.
- Guard is parsed and validated immediately using the condition grammar.

Raises ValueError if the guard is syntactically invalid. The error message includes the transition name.

- Raises ValueError if a transition with the same name already covers any of the same source states (overlap check).
- The parsed guard expression is stored on the Transition object and reused at runtime. The guard string is never re-parsed during sm.send().
- Returns self to support method chaining.

7.2 Event Processing

```
result = sm.send(event, context)
```

Processing steps:

Find all registered transitions where `t.name == event` and `sm.current_state` in `t.from_states`.

If none found: return `TransitionResult(accepted=False, reason="No transition '...' from state '...')"`.

Take the first match (registration order).

If the transition has a guard:

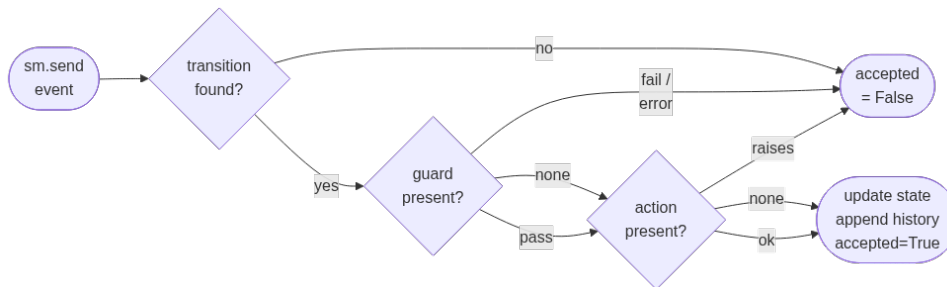
- Evaluate the parsed guard against context.
- If `EvaluationError` (e.g. missing field): return `TransitionResult(accepted=False, reason="Guard error: ...")`.
- If guard is `False`: return `TransitionResult(accepted=False, reason="Guard condition not met: ...")`.

If the transition has an action:

- Call `action(context)`.
- If the action raises any exception: return `TransitionResult(accepted=False, reason="Action error: ...")`. State does not change.

Update `sm.current_state` to `to_state`.
 Append a `HistoryEntry` to `sm.history`.
 Return `TransitionResult(accepted=True, ...)`.

`send()` never raises. All failures are captured in the result.



7.3 can()

```
sm.can(event, context={})
```

Returns True if `send(event, context)` would be accepted at the current state. Does not modify state or history. If guard evaluation raises `EvaluationError`, returns False.

7.4 reset()

```
sm.reset()           # reset to initial state
sm.reset("processing") # reset to a specific state
```

Sets `current_state` to initial (or the given state) and clears history. Does not remove registered transitions.

7.5 states property

Returns a `set[str]` of all states known to the machine: the initial state plus all `from_states` and `to_state` values across all registered transitions.

8. Error Handling

Scenario	When	Behaviour
Invalid guard syntax	<code>sm.transition()</code>	Raises <code>ValueError</code> immediately

Scenario	When	Behaviour
Duplicate (name, from_state)	sm.transition()	Raises ValueError immediately
No matching transition	sm.send()	Returns TransitionResult(accepted=False)
Guard field missing from context	sm.send() evaluation	Returns TransitionResult(accepted=False, reason="Guard error: ...")
Guard evaluates to False	sm.send() evaluation	Returns TransitionResult(accepted=False, reason="Guard condition not met: ...")
Action raises any exception	sm.send() action call	Returns TransitionResult(accepted=False, reason="Action error: ..."). State not updated.

EvaluationError is importable from the top-level package:

```
from state_machine import EvaluationError
```

9. Public API

All public symbols importable from state_machine directly:

```
from state_machine import StateMachine, EvaluationError, TransitionResult, HistoryEntry
```

10. Complete Usage Example

```
from state_machine import StateMachine

sm = StateMachine(initial="draft")
```

```
sm.transition(
    "submit",
    from_state="draft",
    to_state="submitted",
    guard='order.total > 0 AND customer.verified == true',
    description="Submit order for processing.",
)

sm.transition(
    "process",
    from_state="submitted",
    to_state="processing",
    action=lambda ctx: ctx["order"].update({"processing_started": True}),
    description="Begin fulfillment.",
)

sm.transition(
    "ship",
    from_state="processing",
    to_state="shipped",
    guard='order.warehouse_confirmed == true',
    action=lambda ctx: ctx["order"].update({"shipped": True}),
    description="Mark as shipped after warehouse confirmation.",
)

sm.transition(
    "deliver",
    from_state="shipped",
    to_state="delivered",
    description="Mark order as delivered.",
)

sm.transition(
    "cancel",
    from_state=["draft", "submitted", "processing"],
    to_state="cancelled",
    description="Cancel order. Not allowed after shipping.",
)

context = {
    "customer": {"verified": True},
    "order": {"total": 150, "warehouse_confirmed": True},
}

result = sm.send("submit", context)
# result.accepted == True
# result.to_state == "submitted"
# sm.current_state == "submitted"
```

```
result = sm.send("process", context)
# result.accepted == True
# context["order"]["processing_started"] == True

result = sm.send("cancel", context)
# Already in 'processing' -- still cancellable
# result.accepted == True
# sm.current_state == "cancelled"

# Guard rejection
sm.reset()
bad_ctx = {"customer": {"verified": False}, "order": {"total": 0}}
result = sm.send("submit", bad_ctx)
# result.accepted == False
# result.reason == "Guard condition not met: order.total > 0 AND customer.verified == true"

# History
sm2 = StateMachine(initial="draft")
sm2.transition("submit", from_state="draft", to_state="submitted")
sm2.transition("process", from_state="submitted", to_state="processing")
sm2.send("submit", {})
sm2.send("process", {})

sm2.history[0].from_state == "draft"
sm2.history[0].to_state == "submitted"
sm2.history[1].from_state == "submitted"
sm2.history[1].to_state == "processing"
```

11. Out of Scope (v1)

- Named state declarations (states are inferred from transitions)
- Entry and exit actions on states
- State machine serialisation / persistence
- Concurrent or parallel states
- Hierarchical (nested) states
- Transition priority (ties resolved by registration order)
- Async execution
- Thread safety guarantees
- Visual diagram export